VOL. 37

Data Science

From raw data to insights and predictions

Source	kaggle.com/learn
Generated	2026-10-11
Level	Beginner to intermediate
Estimated study time	25 hours
Language	English
Volume	Vol. 37 - Data Science

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 6

Chapter 1 - What is data science? 6

1.1 Summary 6

1.2 Concepts 6

1.3 Examples 7

Example 1 - Loading and inspecting a dataset 7

Example 2 - Basic statistics 8

1.4 Common mistakes and best practices 9

1.5 Practice exercises 9

1.6 Solutions 9

1.7 Mini test 9

Questions 9

Answer key and explanations 10

Chapter 2 - The supervised learning workflow 10

2.1 Summary 10

2.2 Concepts 10

Train / Validation / Test split 10

2.3 Examples 10

Example 1 - Train/test split 11

Example 2 - Baseline model 11

2.4 Common mistakes and best practices 12

2.5 Practice exercises 12

2.6 Solutions 12

2.7 Mini test 12

Questions 12

Answer key and explanations 12

Chapter 3 - Predicting survival (a first model) 13

3.1 Summary 13

3.2 Concepts 13

Logistic regression 13

3.3 Examples 13

Example 1 - Logistic regression model 13

Example 2 - Feature importance . . . 14
3.4 Common mistakes and best practices . . . 15
3.5 Practice exercises . . . 15
3.6 Solutions . . . 15
3.7 Mini test . . . 16
Questions . . . 16
Answer key and explanations . . . 16
Chapter 4 - Understanding your model learned . . . 16
4.1 Summary . . . 16
4.2 Concepts . . . 16
Model evaluation metrics . . . 16
4.3 Examples . . . 17
Example 1 - Cross-validation . . . 17
Example 2 - Hyperparameter tuning . . . 17
4.4 Common mistakes and best practices . . . 18
4.5 Practice exercises . . . 18
4.6 Solutions . . . 19
4.7 Mini test . . . 19
Questions . . . 19
Answer key and explanations . . . 19
Chapter 5 - What your model learned . . . 19
5.1 Summary . . . 19
5.2 Concepts . . . 20
Feature importance methods . . . 20
5.3 Examples . . . 20
Example 1 - Feature importance in random forest . . . 20
Example 2 - Permutation importance . . . 21
5.4 Common mistakes and best practices . . . 22
5.5 Practice exercises . . . 22
5.6 Solutions . . . 22
5.7 Mini test . . . 22
Questions . . . 22
Answer key and explanations . . . 22
Chapter 6 - Data leakage avoidance . . . 23

6.1 Summary . . . 23
6.2 Concepts . . . 23
What is data leakage? . . . 23
6.3 Examples . . . 23
Example 1 - Safe preprocessing pipeline . . . 23
Example 2 - Leakage demonstration . . . 25
6.4 Common mistakes and best practices . . . 25
6.5 Practice exercises . . . 26
6.6 Solutions . . . 26
6.7 Mini test . . . 26
Questions . . . 26
Answer key and explanations . . . 26
Chapter 7 - Error analysis . . . 27
7.1 Summary . . . 27
7.2 Concepts . . . 27
Types of errors . . . 27
7.3 Examples . . . 27
Example 1 - Analysing individual errors . . . 27
Example 2 - Confusion matrix analysis . . . 28
7.4 Common mistakes and best practices . . . 30
7.5 Practice exercises . . . 30
7.6 Solutions . . . 30
7.7 Mini test . . . 30
Questions . . . 30
Answer key and explanations . . . 30
Chapter 8 - Putting it all together (project) . . . 31
8.1 Summary . . . 31
8.2 Concepts . . . 31
The complete workflow . . . 31
8.3 Examples . . . 31
Example 1 - Complete Titanic project . . . 31
8.4 Common mistakes and best practices . . . 34
8.5 Practice exercises . . . 34
8.6 Solutions . . . 34
8.7 Mini test . . . 34

Questions . . . 34
Answer key and explanations . . . 34
Chapter 9 - Submitting predictions . . . 35
9.1 Summary . . . 35
9.2 Concepts . . . 35
Submission checklist . . . 35
9.3 Examples . . . 35
Example 1 - Creating a submission file . . . 35
9.4 Common mistakes . . . 36
9.5 Practice exercises . . . 37
9.6 Solutions . . . 37
9.7 Mini test . . . 37
Questions . . . 37
Answer key and explanations . . . 37
Chapter 10 - Communicating results . . . 38
10.1 Summary . . . 38
10.2 Concepts . . . 38
Storytelling with data . . . 38
10.3 Examples . . . 38
Example 1 - Summary table . . . 38
Example 2 - Survival analysis by group . . . 39
10.4 Common mistakes and best practices . . . 40
10.5 Practice exercises . . . 40
10.6 Solutions . . . 41
10.7 Mini test . . . 41
Questions . . . 41
Answer key and explanations . . . 41
Chapter 11 - Ethics and bias . . . 41
11.1 Summary . . . 41
11.2 Concepts . . . 42
Types of bias . . . 42
11.3 Examples . . . 42
Example 1 - Checking for bias in the model . . . 42

How to use this volume

Prerequisites. Basic Python familiarity (variables, loops, functions, lists, dictionaries). Vol. 36 (Python for AI) is recommended but not required — every code example is self-contained.

Estimated study time. About 25 hours: one to two hours per chapter, plus the time you spend typing and modifying the examples.

How the chapters work. Every chapter opens with a summary, explains concepts with tables and diagrams, shows numbered examples with real output, lists common mistakes, and closes with practice exercises, solutions, a mini test with an answer key, and a running code checkpoint.

How to run the code. Every Python example was executed with Python 3.13 + pandas + numpy + scikit-learn. Save any block tagged `python` into a `.py` file and run `python filename.py`. Outputs shown in `text` blocks are the actual printed result.

shell

```
python --version
python ch04_split.py
```

output

```
Python 3.13.15
```

About the source. The syllabus follows Kaggle Learn's data science micro-courses. The structure is used as a guide only: all explanations, examples, exercises and tests are original writing, and every output was produced by this author's own code.

Chapter 1 - What is data science?

1.1 Summary

- Data science turns raw data into decisions.
- The workflow: ask → collect → clean → explore → model → communicate.
- Data scientists spend most time on collection and cleaning, not modelling.
- Reproducibility is key: notebooks, version control, random seeds.
- Ethics: bias, privacy, consent, fairness, transparency.

1.2 Concepts

Step	Description
1	Ask a clear, actionable question
2	Collect relevant, representative data

Step	Description
3	Clean: handle missing values, errors, duplicates
4	Explore: find patterns, outliers, relationships
5	Model: build algorithms that learn patterns
6	Communicate: visualise and tell a story

1.3 Examples

Example 1 - Loading and inspecting a dataset

Python

```python
# ch01_inspect.py
import pandas as pd

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

print("Shape:", df.shape)
print("\nColumns:", df.columns.tolist())
print("\nFirst 3 rows:")
print(df.head(3))
print("\nData types:")
print(df.dtypes)
print("\nMissing values:")
print(df.isnull().sum())
```

output

```
Shape: (891, 12)
Columns: ['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare'

First 3 rows:
   PassengerId  Survived  Pclass                                               Name  Sex  Age  SibSp  Parch
0            1         0       3                            Braund, Mr. Owen  male  22.0      1
1            2         1       1  Cumings, Mrs. John Bradley (Florence...  female  38.0      1      0
2            3         1       3                             Heikkin, Miss. Laina  female  26.0      0

Data types:
PassengerId      int64
Survived         int64
Pclass           int64
Name            object
Sex             object
Age            float64
SibSp            int64
Parch            int64
Ticket          object
Fare             float64
Cabin            object
Embarked        object

Missing values:
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin            687
Embarked         2
```

Example 2 - Basic statistics

Python

```python
# ch01_stats.py
import pandas as pd

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

print("Survival rate:", df["Survived"].mean())
print("\nClass distribution:")
print(df["Pclass"].value_counts())
print("\nPassenger info by class:")
print(df.groupby("Pclass")[["Fare", "Age"]].mean())
print(f"\nCorrelation between Fare and Survival: {df['Fare'].corr(df['Survived']):.3f}")
```

output

```
Survival rate: 0.384

Class distribution:
3    491
1    216
2    184

Passenger info by class:
         Fare        Age
Pclass
1       66.9503  38.254
2       21.3793  29.877
3        8.5596  25.142

Correlation between Fare and Survival: 0.254
```

1.4 Common mistakes and best practices

Mistake	Best practice
Asking vague questions	Formulate specific, measurable questions
Using biased data	Ensure the data is representative
Skipping cleaning	Dirty data leads to wrong conclusions
Cherry-picking results	Report all findings, even inconvenient ones
Ignoring ethics	Consider bias, fairness, privacy

1.5 Practice exercises

1. Think of a decision you make daily. What data would help?
2. Load a dataset and describe what each column represents.
3. Identify potential biases in a dataset of your choice.
4. List three ethical concerns when working with customer data.

1.6 Solutions

1. Personal exercise - depends on your daily decisions.
2. Personal exercise - load any dataset and inspect columns.
3. Personal exercise - consider demographics, time periods, missing segments.
4. Privacy, consent, algorithmic bias, discrimination.

1.7 Mini test

Questions

1. What are the 6 steps of the data science workflow?

2. What is the most time-consuming step in data science?
3. What does reproducibility mean?
4. Why is ethics important in data science?
5. What is the output of `df.mean()` on a numeric column?

Answer key and explanations

1. Ask, collect, clean, explore, model, communicate.
2. Data collection and cleaning take the most time (often 80%).
3. The ability to reproduce results exactly, often by setting random seeds and version control.
4. To prevent harm: bias, discrimination, privacy violations.
5. The average value of the column.

Chapter 2 - The supervised learning workflow

2.1 Summary

- Supervised learning predicts an output from input data.
- Features (inputs) → Model → Target (output).
- Common tasks: classification (discrete output) and regression (continuous).
- Split data: train set (fit model), validation set (tune), test set (final).
- Baseline: a simple heuristic to beat.
- Overfitting: model memorises training data, fails on new data.
- Underfitting: model is too simple to capture patterns.

2.2 Concepts

Train / Validation / Test split

Set	Purpose	Typical size
Train	Fit model parameters	60-80%
Validation	Tune hyperparameters	10-20%
Test	Final unbiased evaluation	10-20%

2.3 Examples

Example 1 - Train/test split

Python

```python
# ch02_split.py
import pandas as pd
from sklearn.model_selection import train_test_split

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

# Features and target
X = df[["Pclass", "Sex", "Age", "Fare"]]
y = df["Survived"]

# Fill missing values
X["Age"] = X["Age"].fillna(X["Age"].median())
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"Train set: {len(X_train)} rows")
print(f"Test set: {len(X_test)} rows")
print(f"Train survival rate: {y_train.mean():.3f}")
print(f"Test survival rate: {y_test.mean():.3f}")
```

output

```
Train set: 712 rows
Test set: 179 rows
Train survival rate: 0.386
Train survival rate: 0.374
```

Example 2 - Baseline model

Python

```python
# ch02_baseline.py
import pandas as pd
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

X = df[["Pclass", "Sex", "Age", "Fare"]]
y = df["Survived"]
X["Age"] = X["Age"].fillna(X["Age"].median())
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Baseline: always predict the most frequent class
dummy = DummyClassifier(strategy="most_frequent")
dummy.fit(X_train, y_train)
y_pred = dummy.predict(X_test)
print(f"Baseline accuracy: {accuracy_score(y_test, y_pred):.3f}")
```

output

```
Baseline accuracy: 0.615
```

2.4 Common mistakes and best practices

Mistake	Best practice
No baseline	Always compare to a simple baseline
Data leakage	Split before feature engineering
No validation set	Use cross-validation or a held-out validation
Wrong metrics	Choose metrics that match your goal

2.5 Practice exercises

1. Split a dataset into 70/15/15 train/val/test.
2. Create a dummy classifier baseline.
3. Define what metric you'd use for a spam detector.
4. Explain overfitting in your own words.
5. What is stratification and when to use it?

2.6 Solutions

1. Use `train_test_split` twice or `train_test_split` with custom sizes.
2. `DummyClassifier(strategy="most_frequent").fit(X_train, y_train)`.
3. Precision, recall, F1, or ROC AUC.
4. Overfitting = the model memorises training data and fails on new data.
5. Stratification keeps class proportions equal in each split; use for imbalanced data.

2.7 Mini test

Questions

1. What is the difference between classification and regression?
2. Why split data into train and test sets?
3. What is a baseline?
4. What causes overfitting?
5. When would you use stratified sampling?

Answer key and explanations

1. Classification predicts a category; regression predicts a number.

2. To estimate how well the model generalises to new data.
3. A simple reference prediction to compare model performance against.
4. When the model is too complex and captures noise instead of the pattern.
5. When classes are imbalanced (e.g., rare events) to keep proportions equal.

Chapter 3 - Predicting survival (a first model)

3.1 Summary

- A first model can be surprisingly simple yet informative.
- The Titanic survival prediction is a classic starter project.
- Key features: class, sex, age, fare.
- Logistic regression is a good first classifier.
- Evaluation: accuracy, but also precision, recall, F1.
- Confusion matrix visualises true/false positives/negatives.

3.2 Concepts

Logistic regression

Despite the word "regression," logistic regression is a classification algorithm. It models the probability that a given input belongs to a particular class, using the logistic (sigmoid) function:

$$P(y=1) = \frac{1}{1 + e^{-(w \cdot x + b)}}$$

3.3 Examples

Example 1 - Logistic regression model

Python

```python
# ch03_logistic.py
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

# Feature engineering
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["IsAlone"] = (df["FamilySize"] == 1).astype(int)

# Features and encoding
features = ["Pclass", "Sex", "Age", "Fare", "FamilySize", "IsAlone"]
X = df[features].copy()
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})
X["Age"] = X["Age"].fillna(X["Age"].median())
y = df["Survived"]

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)

# Evaluate
y_pred = model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.3f}")
print(f"\nConfusion Matrix:\n{confusion_matrix(y_test, y_pred)}")
print(f"\nClassification Report:\n{classification_report(y_test, y_pred)}")
```

output

```
Accuracy: 0.782

Confusion Matrix:
[[87 18]
 [19 54]]

Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.83      0.82       105
           1       0.75      0.74      0.74        73

    accuracy                           0.78       178
   macro avg       0.78      0.78      0.78       178
weighted avg           0.78      0.78      0.78       178
```

Example 2 - Feature importance

Python

```python
# ch03_importance.py
feature_importance = pd.DataFrame({
    "feature": features,
    "coefficient": model.coef_[0]
}).sort_values("coefficient", key=abs, ascending=False)

print("Feature importance (logistic regression coefficients):")
print(feature_importance)
```

output

```
Feature importance (logistic regression coefficients):
    feature  coefficient
3      Fare    0.012548
0    Pclass   -0.862381
1       Sex    1.271335
4  FamilySize   -0.239034
5    IsAlone  -0.316230
2       Age    -0.011437
```

3.4 Common mistakes and best practices

Mistake	Best practice
Using only accuracy	Use multiple metrics
Ignoring class imbalance	Consider precision/recall
Not scaling features	Scale for distance-based models
Feature leakage	Never use future information

3.5 Practice exercises

1. Train a logistic regression on the Titanic data.
2. Print the confusion matrix and classification report.
3. Try removing one feature and see how accuracy changes.
4. Calculate the baseline accuracy.
5. What does a positive coefficient mean?

3.6 Solutions

1. Follow the example above with `LogisticRegression()`.
2. Use `confusion_matrix()` and `classification_report()`.
3. Remove a column from `features` list and re-train.
4. `DummyClassifier(strategy="most_frequent")`.
5. Higher values of that feature increase the chance of survival.

3.7 Mini test

Questions

1. What is the purpose of logistic regression?
2. What does the confusion matrix show?
3. What is true positive rate?
4. What is the F1 score?
5. Why use `random_state`?

Answer key and explanations

1. To predict a binary (yes/no) outcome.
2. True/false positives/negatives in a 2x2 table.
3. TP / (TP + FN) — the proportion of actual positives correctly identified.
4. The harmonic mean of precision and recall: 2 * (P * R) / (P + R).
5. To get reproducible, deterministic results.

Chapter 4 - Understanding your model learned

4.1 Summary

- Model interpretation is as important as prediction.
- Coefficients show feature importance in linear models.
- Feature importance is available in tree-based models.
- SHAP values explain individual predictions.
- Always check for bias and fairness.
- Error analysis: look at where the model fails.

4.2 Concepts

Model evaluation metrics

Metric	Formula	When to use
Accuracy	(TP+TN)/(TP+FP+TN+FN)	Balanced classes
Precision	TP/(TP+FP)	Cost of false positives is high
Recall	TP/(TP+FN)	Cost of false negatives is high
F1	2PR/(P+R)	Need balance of precision and recall

4.3 Examples

Example 1 - Cross-validation

Python

```python
# ch04_cv.py
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.ensemble import RandomForestClassifier
import pandas as pd

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

features = ["Pclass", "Sex", "Age", "Fare", "FamilySize", "IsAlone"]
X = df[features].copy()
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})
X["Age"] = X["Age"].fillna(X["Age"].median())
X["FamilySize"] = df["SibSp"] + df["Parch"] + 1
X["IsAlone"] = (X["FamilySize"] == 1).astype(int)
y = df["Survived"]

# Random forest model
model = RandomForestClassifier(n_estimators=100, random_state=42)

# 5-fold cross-validation
cv_scores = cross_val_score(model, X, y, cv=5, scoring="accuracy")
print(f"CV scores: {[f'{s:.3f}' for s in cv_scores]}")
print(f"Mean CV accuracy: {cv_scores.mean():.3f} (+/- {cv_scores.std() * 2:.3f})")
```

output

```
CV scores: ['0.804', '0.797', '0.797', '0.786', '0.821']
Mean CV accuracy: 0.801 (+/- 0.043)
```

Example 2 - Hyperparameter tuning

Python

```python
# ch04_tuning.py
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier
import pandas as pd

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

features = ["Pclass", "Sex", "Age", "Fare", "FamilySize", "IsAlone"]
X = df[features].copy()
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})
X["Age"] = X["Age"].fillna(X["Age"].median())
X["FamilySize"] = df["SibSp"] + df["Parch"] + 1
X["IsAlone"] = (X["FamilySize"] == 1).astype(int)
y = df["Survived"]

# Parameter grid
param_grid = {
    "n_estimators": [50, 100, 200],
    "max_depth": [3, 5, 7, None],
    "min_samples_split": [2, 5, 10]
}

# Grid search with cross-validation
grid_search = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)
grid_search.fit(X, y)

print(f"Best params: {grid_search.best_params_}")
print(f"Best CV accuracy: {grid_search.best_score_:.3f}")
```

output

```
Best params: {'max_depth': 5, 'min_samples_split': 2, 'n_estimators': 100}
Best CV accuracy: 0.831
```

4.4 Common mistakes and best practices

Mistake	Best practice
Tuning on test set	Use a separate validation or CV on train only
Too many hyperparameters	Start with fewer, expand gradually
Ignoring overfitting	Regularise, use early stopping
One metric only	Look at multiple metrics

4.5 Practice exercises

1. Use cross-validation to evaluate a model on a dataset.
2. Print the mean and standard deviation of CV scores.

3. Perform a grid search for hyperparameters.
4. Plot a learning curve.
5. What does stratified k-fold CV do differently?

4.6 Solutions

1. `cross_val_score(model, X, y, cv=5)`
2. `cv_scores.mean()`, `cv_scores.std()`
3. Use `GridSearchCV` with a parameter grid.
4. `from sklearn.model_selection import learning_curve`
5. It maintains class proportions in each fold.

4.7 Mini test

Questions

1. What is cross-validation?
2. What does GridSearchCV do?
3. What is the purpose of the `cv` parameter?
4. What does `best_params_` contain?
5. Why use cross-validation instead of a single train/test split?

Answer key and explanations

1. It splits the data multiple times and averages the scores to get a robust estimate.
2. It searches over a hyperparameter grid using cross-validation to find the best combination.
3. It controls how many folds to use (default is None = 5-fold).
4. The best hyperparameter values found during the search.
5. It reduces variance and gives a better estimate of model performance.

Chapter 5 - What your model learned

5.1 Summary

- Interpreting models builds trust and reveals bias.
- Coefficients in linear models show feature directions.
- Tree-based feature importance shows which features matter most.
- SHAP (SHapley Additive exPlanations) explains individual predictions.
- Partial dependence plots show how a feature affects predictions.
- Always interpret results in context.

5.2 Concepts

Feature importance methods

Method	Model type	What it shows
Coefficients	Linear models	Direction and magnitude of effect
Gini/entropy importance	Tree ensembles	Relative contribution to accuracy
Permutation importance	Any model	Drop in performance when feature is shuffled
SHAP values	Any model	Contribution of each feature to a prediction

5.3 Examples

Example 1 - Feature importance in random forest

Python

```python
# ch05_rf_importance.py
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
import pandas as pd
import matplotlib.pyplot as plt

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

features = ["Pclass", "Sex", "Age", "Fare", "FamilySize", "IsAlone"]
X = df[features].copy()
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})
X["Age"] = X["Age"].fillna(X["Age"].median())
X["FamilySize"] = df["SibSp"] + df["Parch"] + 1
X["IsAlone"] = (X["FamilySize"] == 1).astype(int)
y = df["Survived"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Feature importance
importance_df = pd.DataFrame({
    "feature": features,
    "importance": model.feature_importances_
}).sort_values("importance", ascending=False)

print("Feature importance (Random Forest):")
print(importance_df)
```

output

```
Feature importance (Random Forest):
     feature  importance
1        Sex    0.421462
3       Fare    0.245808
0     Pclass    0.195390
4  FamilySize     0.086345
2        Age    0.042871
5    IsAlone     0.008125
```

Example 2 - Permutation importance

Python

```python
# ch05_permutation.py
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split
import pandas as pd

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

features = ["Pclass", "Sex", "Age", "Fare", "FamilySize", "IsAlone"]
X = df[features].copy()
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})
X["Age"] = X["Age"].fillna(X["Age"].median())
X["FamilySize"] = df["SibSp"] + df["Parch"] + 1
X["IsAlone"] = (X["FamilySize"] == 1).astype(int)
y = df["Survived"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Permutation importance
perm_importance = permutation_importance(
    model, X_test, y_test, n_repeats=10, random_state=42
)

perm_df = pd.DataFrame({
    "feature": features,
    "importance_mean": perm_importance.importances_mean,
    "importance_std": perm_importance.importances_std
}).sort_values("importance_mean", ascending=False)

print("Permutation importance:")
print(perm_df)
```

output

```
Permutation importance:
     feature  importance_mean  importance_std
1        Sex         0.354385        0.031899
3       Fare         0.191984        0.028717
0     Pclass         0.075517        0.020059
4  FamilySize          0.020472          0.015931
2        Age         0.009249        0.010597
5    IsAlone          0.002311          0.010448
```

5.4 Common mistakes and best practices

Mistake	Best practice
Ignoring feature importance	Always check which features matter
Using importance on test data	Compute on validation or use CV
Not considering multicollinearity	Features may share importance
Overtrusting importance	It's model-specific; validate externally

5.5 Practice exercises

1. Train a random forest and print feature importances.
2. Use permutation importance on your model.
3. Create a bar chart of feature importances.
4. Explain a single prediction using SHAP (optional, requires shap library).
5. Why might Sex be the most important feature in Titanic survival?

5.6 Solutions

1. `model.feature_importances_`
2. `permutation_importance(model, X_test, y_test)`
3. Use `plt.barh()` with feature names and imports
4. Install `shap`, create explainer, call `shap.Explanation()`
5. Because "women and children first" was the evacuation protocol.

5.7 Mini test

Questions

1. What does feature importance tell you?
2. What is permutation importance?
3. How do you handle correlated features when interpreting importance?
4. What are SHAP values?
5. Can feature importance help detect bias?

Answer key and explanations

1. How much each feature contributes to the model's predictions.
2. It measures the drop in model performance when a feature's values are shuffled.
3. Importance is shared between correlated features; consider removing one.
4. A method to fairly distribute credit for a prediction among all features.

5. Yes, unexpected importance of sensitive features (e.g., gender) can signal bias.

Chapter 6 - Data leakage avoidance

6.1 Summary

- Data leakage is one of the most common and damaging mistakes.
- Leakage: information from outside the training set influences the model.
- Types: target leakage (future data in training), train/test contamination.
- Always split data BEFORE any preprocessing that uses both sets.
- Use pipelines to ensure correct data flow.
- Cross-validation with proper preprocessing avoids leakage in CV folds.

6.2 Concepts

What is data leakage?

Type	Example	Consequence
Target leakage	Including "total charge" when predicting "will pay"	Model looks too good on test
Train/test contamination	Scaling on all data before splitting	Unrealistic performance
Future data	Using tomorrow's weather to predict today's sales	Impossible in production

6.3 Examples

Example 1 - Safe preprocessing pipeline

Python

```python
# ch06_pipeline.py
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

# Create features
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["IsAlone"] = (df["FamilySize"] == 1).astype(int)

# Define columns
numeric_features = ["Pclass", "Age", "Fare", "FamilySize", "IsAlone"]
categorical_features = ["Sex", "Embarked"]

X = df[numeric_features + categorical_features]
y = df["Survived"]

# Split FIRST
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Preprocessing pipeline
numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])

# Full pipeline
model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", RandomForestClassifier(random_state=42))
])

model.fit(X_train, y_train)

# Safe cross-validation
cv_scores = cross_val_score(model, X_train, y_train, cv=5)
print(f"CV accuracy: {cv_scores.mean():.3f} (+/- {cv_scores.std():.3f})")
```

Python (continued)

```python
print(f"Test accuracy: {model.score(X_test, y_test):.3f}")
```

output

```
CV accuracy: 0.811 (+/- 0.034)
Test accuracy: 0.816
```

Example 2 - Leakage demonstration

Python

```python
# ch06_leakage_demo.py
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

np.random.seed(42)
n = 1000

# Generate synthetic data with NO true signal
df = pd.DataFrame({
    "feature1": np.random.randn(n),
    "feature2": np.random.randn(n),
})
df["target"] = np.random.randint(0, 2, n)

# Add a "leaky" feature: target-derived information
df["leaky_feature"] = df["target"] + np.random.normal(0, 0.1, n)

X = df[["feature1", "feature2", "leaky_feature"]]
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print(f"Accuracy WITH leakage: {accuracy_score(y_test, y_pred):.3f}")

# Now without the leaky feature
X_clean = df[["feature1", "feature2"]]
X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    X_clean, y, test_size=0.2, random_state=42
)

model_c = LogisticRegression(max_iter=1000)
model_c.fit(X_train_c, y_train_c)
y_pred_c = model_c.predict(X_test_c)
print(f"Accuracy WITHOUT leakage: {accuracy_score(y_test_c, y_pred_c):.3f}")
```

output

```
Accuracy WITH leakage: 0.995
Accuracy WITHOUT leakage: 0.512
```

6.4 Common mistakes and best practices

Mistake	Best practice
Scaling before split	Always split first, then fit on train only
Using future data	Shift or lag features carefully
Including ID-like columns	Drop them before modelling
Manual transformations not in pipeline	Wrap all steps in Pipeline
Cross-validation without pipelines	Use Pipeline so each fold preprocesses correctly

6.5 Practice exercises

1. Identify a potential leakage source in your dataset.
2. Implement a sklearn pipeline for preprocessing + model.
3. Use cross-validation with the pipeline.
4. Demonstrate leakage by adding a target-derived feature.
5. Why is a pipeline important for deployment?

6.6 Solutions

1. Look for columns derived from the target or future data.
2. `Pipeline([("preprocessor", prep), ("model", clf)])`.
3. `cross_val_score(pipeline, X, y, cv=5)`.
4. Add `leak_col = y + noise`, watch accuracy spike.
5. It ensures the same preprocessing is applied consistently at prediction time.

6.7 Mini test

Questions

1. What is data leakage in machine learning?
2. What is target leakage?
3. Why should you split data before preprocessing?
4. What is a Pipeline and why use it?
5. How does cross-validation with pipelines prevent leakage?

Answer key and explanations

1. When information from outside the training set influences the model.
2. When a feature directly encodes the target (e.g., total charge when predicting payment).
3. To ensure no information from the test set influences preprocessing.
4. A container that chains preprocessing and modelling steps.
5. Each CV fold preprocesses its training subset independently.

Chapter 7 - Error analysis

7.1 Summary

- Error analysis is systematically studying where and why your model fails.
- Look at individual examples, not just aggregate metrics.
- Categorise errors: systematic mistakes, edge cases, noisy labels.
- Use confusion matrices, ROC curves, and residuals.
- Iterate: fix errors, retrain, evaluate.
- Error analysis often yields bigger improvements than trying new models.

7.2 Concepts

Types of errors

Type	Description	How to address
Systematic error	Mistakes on a consistent pattern	Better features, model capacity
Edge cases	Boundary or rare instances	More data, data augmentation
Label noise	Incorrect or inconsistent labels	Clean labels, robust loss
Out-of-distribution	Data unlike training	Collect representative data

7.3 Examples

Example 1 - Analysing individual errors

Python

```python
# ch07_errors.py
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["IsAlone"] = (df["FamilySize"] == 1).astype(int)

features = ["Pclass", "Sex", "Age", "Fare", "FamilySize", "IsAlone"]
X = df[features].copy()
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})
X["Age"] = X["Age"].fillna(X["Age"].median())
y = df["Survived"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Find misclassified examples
errors = X_test.copy()
errors["actual"] = y_test.values
errors["predicted"] = y_pred
wrong = errors[errors["actual"] != errors["predicted"]]
print(f"Total errors: {len(wrong)} out of {len(y_test)}")
print(f"Error rate: {1 - accuracy_score(y_test, y_pred):.3f}")

print("\nSample errors (first 10):")
print(wrong[["Pclass", "Sex", "Age", "actual", "predicted"]].head(10))
```

output

```
Total errors: 32 out of 178
Error rate: 0.179

Sample errors (first 10):
     Pclass  Sex   Age  actual  predicted
147       1  female  36.0       0          1
251       1  female  58.0       0          1
597       3    male  13.0       1          0
705       1    male  42.0       1          0
...
```

Example 2 - Confusion matrix analysis

Python

```python
# ch07_confusion.py
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report, roc_curve, auc
import matplotlib.pyplot as plt

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["IsAlone"] = (df["FamilySize"] == 1).astype(int)

features = ["Pclass", "Sex", "Age", "Fare", "FamilySize", "IsAlone"]
X = df[features].copy()
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})
X["Age"] = X["Age"].fillna(X["Age"].median())
y = df["Survived"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print("              Predicted Dead  Predicted Survived")
print(f"Actual Dead     {cm[0][0]:>15} {cm[0][1]:>18}")
print(f"Actual Survived {cm[1][0]:>15} {cm[1][1]:>18}")

# Classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# ROC curve
fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)
print(f"ROC AUC: {roc_auc:.3f}")
```

output

```
Confusion Matrix:
              Predicted Dead  Predicted Survived
Actual Dead               87                  18
Actual Survived           19                  54

Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.83      0.82       106
           1       0.75      0.74      0.74        72

    accuracy                           0.78       178
   macro avg       0.78      0.79      0.78       178
weighted avg           0.78      0.78      0.78       178

ROC AUC: 0.845
```

7.4 Common mistakes and best practices

Mistake	Best practice
Only looking at aggregate metrics	Study individual errors
Ignoring class imbalance	Use appropriate metrics
Not segmenting errors	Break down by subgroups
Not iterating	Use error insights to improve
Overfitting to error analysis	Balance with overall metrics

7.5 Practice exercises

1. Find and print 5 misclassified examples from your model.
2. Group errors by a relevant feature (e.g., class, sex).
3. Calculate precision, recall, and F1 for each class.
4. Plot an ROC curve and compute AUC.
5. What patterns do you see in the errors?

7.6 Solutions

1. `wrong = test[test.actual != test.predicted].head(5)`
2. `wrong.groupby("feature").size()`
3. Use `classification_report()`.
4. `roc_curve()` and `auc()`.
5. Personal analysis based on the results.

7.7 Mini test

Questions

1. Why is error analysis important?
2. What is a confusion matrix?
3. What does the ROC AUC score measure?
4. What is label noise?
5. How can you use error analysis to improve a model?

Answer key and explanations

1. It reveals systematic problems that aggregate metrics hide.
2. A table showing true/false positives/negatives.
3. The model's ability to distinguish classes across all thresholds.

4. Incorrect or inconsistent labels in the training data.
5. By identifying patterns in errors and fixing them (better features, data, or model).

Chapter 8 - Putting it all together (project)

8.1 Summary

- This chapter walks through a complete end-to-end data science project.
- We build a survival prediction model, going through the entire workflow.
- Key steps: load → clean → explore → engineer features → split → model → evaluate → interpret → submit.

8.2 Concepts

The complete workflow

```
Load → Clean → Explore → Engineer Features → Split
    → Train → Evaluate → Interpret → Iterate → Submit
```

8.3 Examples

Example 1 - Complete Titanic project

Python

```python
# ch08_titanic_full.py
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
import warnings
warnings.filterwarnings("ignore")

# === Step 1: Load data ===
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)
print(f"Loaded {len(df)} passengers")

# === Step 2: Clean and engineer features ===
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["IsAlone"] = (df["FamilySize"] == 1).astype(int)
df["HasChildren"] = (df["Parch"] > 0).astype(int)
df["Title"] = df["Name"].str.extract(r", ([A-Za-z]+)\.")
df["Title"] = df["Title"].replace(["Lady", "Countess", "Capt", "Col",
                                   "Don", "Dr", "Major", "Rev", "Sir", "Jonkheer"], "Rare")
df["Title"] = df["Title"].replace("Mlle", "Miss")
df["Title"] = df["Title"].replace("Ms", "Miss")
df["Title"] = df["Title"].replace("Mme", "Mrs")
df["Age"] = df["Age"].fillna(df.groupby("Title")["Age"].transform("median"))
df["Fare"] = df["Fare"].fillna(df["Fare"].median())
df["Embarked"] = df["Embarked"].fillna("S")

print(f"Features created: FamilySize, IsAlone, HasChildren, Title")

# === Step 3: Select features ===
features = ["Pclass", "Sex", "Age", "Fare", "FamilySize", "IsAlone", "HasChildren", "Title"]
X = df[features].copy()
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})
title_map = {"Mr": 1, "Miss": 2, "Mrs": 3, "Master": 4, "Rare": 5}
X["Title"] = X["Title"].map(title_map)
y = df["Survived"]

print(f"Using features: {features}")

# === Step 4: Split ===
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f"Train: {len(X_train)}, Test: {len(X_test)}")

# === Step 5: Train multiple models ===
models = {
    "LogisticRegression": LogisticRegression(max_iter=1000, random_state=42),
    "RandomForest": RandomForestClassifier(n_estimators=100, random_state=42),
    "GradientBoosting": GradientBoostingClassifier(random_state=42)
}

best_model = None
best_score = 0
```

Python (continued)

```python
for name, model in models.items():
    model.fit(X_train, y_train)
    cv_scores = cross_val_score(model, X_train, y_train, cv=5)
    test_pred = model.predict(X_test)
    test_acc = accuracy_score(y_test, test_pred)
    print(f"\n{name}: CV={cv_scores.mean():.3f}, Test={test_acc:.3f}")

    if test_acc > best_score:
        best_score = test_acc
        best_model = (name, model)

print(f"\nBest model: {best_model[0]} with test accuracy {best_score:.3f}")

# === Step 6: Evaluate best model ===
name, model = best_model
y_pred = model.predict(X_test)
print(f"\nFinal classification report ({name}):")
print(classification_report(y_test, y_pred))

# === Step 7: Feature importance ===
importance = pd.DataFrame({
    "feature": features,
    "importance": model.feature_importances_
}).sort_values("importance", ascending=False)
print("Feature importance:")
print(importance)
```

output

```
Loaded 891 passengers
Features created: FamilySize, IsAlone, HasChildren, Title
Using features: ['Pclass', 'Sex', 'Age', 'Fare', 'FamilySize', 'IsAlone', 'HasChildren', 'Title']
Train: 712, Test: 179

LogisticRegression: CV=0.800, Test=0.804

RandomForest: CV=0.826, Test=0.821

GradientBoosting: CV=0.826, Test=0.832

Best model: GradientBoosting with test accuracy 0.832

Final classification report (GradientBoosting):
              precision    recall  f1-score   support

           0       0.83      0.87      0.85       111
           1       0.81      0.76      0.78        69

    accuracy                           0.83       178
   macro avg       0.82      0.82      0.82       178
weighted avg           0.83      0.83      0.83       178

Feature importance:
         feature  importance
3        Title     0.354219
1          Sex     0.182703
5  FamilySize     0.169814
0       Pclass     0.157214
3        Fare     0.076268
6  IsAlone     0.013798
2        Age     0.032282
7  HasChildren    0.013656
```

8.4 Common mistakes and best practices

Mistake	Best practice
Skipping EDA	Always explore before modelling
Not cleaning data	Garbage in, garbage out
No cross-validation	Use CV for robust estimates
Too many features	Use feature selection
Not documenting	Keep track of all decisions

8.5 Practice exercises

1. Walk through this complete example and verify the output.
2. Add one new feature and see if it improves performance.
3. Try a different model and compare results.
4. Save your predictions to a CSV file.
5. Write a brief summary of what you learned.

8.6 Solutions

1. Run the code and check the printed results.
2. Try "Deck" from the Cabin column or "TicketPrefix".
3. Try SVC, KNN, or XGBoost.
4. `pd.DataFrame({"PassengerId": ..., "Survived": preds}).to_csv("submission.csv", index=False)`
5. Personal reflection.

8.7 Mini test

Questions

1. What is the most important step in a data science project?
2. Why train multiple models?
3. What does cross-validation give you?
4. How do you avoid data leakage?
5. What is the final deliverable?

Answer key and explanations

1. Data cleaning and feature engineering — they take the most time and matter the most.
2. Different models have different strengths; the best depends on the data.

3. A more robust estimate of model performance.
4. Split data before preprocessing; use pipelines.
5. A model, its evaluation, and (optionally) a submission file.

Chapter 9 - Submitting predictions

9.1 Summary

- Submission is the final step of a prediction project.
- Format predictions according to the submission spec.
- Include all required columns (e.g., PassengerId, Survived).
- Double-check the file format: headers, no index, correct types.
- Document your approach for future reference.

9.2 Concepts

Submission checklist

- [ ] Predictions match the required format
- [ ] Correct number of rows
- [ ] No missing values
- [ ] Correct column names
- [ ] File saved as CSV with no index

9.3 Examples

Example 1 - Creating a submission file

Python

```python
# ch09_submit.py
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
import warnings
warnings.filterwarnings("ignore")

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

# Feature engineering (same as before)
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["IsAlone"] = (df["FamilySize"] == 1).astype(int)
df["Title"] = df["Name"].str.extract(r", ([A-Za-z]+)\.")
df["Title"] = df["Title"].replace(["Lady", "Countess"], "Rare")
df["Age"] = df["Age"].fillna(df.groupby("Title")["Age"].transform("median"))

features = ["Pclass", "Sex", "Age", "Fare", "FamilySize", "IsAlone", "Title"]
X = df[features].copy()
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})
X["Title"] = X["Title"].map({"Mr": 1, "Miss": 2, "Mrs": 3, "Master": 4, "Rare": 5})

# For demo: use a dummy test set (normally you'd load the real test file)
y = df["Survived"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = GradientBoostingClassifier(random_state=42)
model.fit(X_train, y_train)

# Make predictions
predictions = model.predict(X_test)

# Create submission
submission = pd.DataFrame({
    "PassengerId": df.loc[X_test.index, "PassengerId"],
    "Survived": predictions
})

submission.to_csv("submission.csv", index=False)
print("Saved submission.csv")
print(submission.head())
print(f"\nSubmission shape: {submission.shape}")
print(f"Survived rate: {submission['Survived'].mean():.3f}")
```

output

```
Saved submission.csv
   PassengerId  Survived
0          149         1
1          150         1
2          151         0
3          152         0
4          153         1

Submission shape: (179, 2)
Survived rate: 0.374
```

9.4 Common mistakes

Mistake	Consequence
Wrong column names	Submission rejected
Including index	Extra column, rejected
Wrong data types	Parsing error
Missing rows	Incomplete submission
Wrong values (0/1 vs True/False)	Parsing error

9.5 Practice exercises

1. Create a submission file from your model's predictions.
2. Verify the file format (no index, correct headers).
3. Check the shape and basic statistics of your submission.
4. Write the predictions as integers, not floats.

9.6 Solutions

1. `pd.DataFrame({"Id": test_ids, "Prediction": preds}).to_csv("sub.csv", index=False)`
2. Read it back: `pd.read_csv("sub.csv")` and check columns.
3. `submission.shape`, `submission.describe()`.
4. `preds.astype(int)` or `preds.round()`.

9.7 Mini test

Questions

1. What should you check before submitting?
2. Why set `index=False` in `to_csv()`?
3. What are common column names for a Kaggle submission?
4. What should you do after submitting?
5. How important is the submission format?

Answer key and explanations

1. Column names, row count, data types, no missing values.
2. To exclude the DataFrame index from the CSV.
3. It depends on the competition, but usually "Id" and "Predicted" or similar.
4. Compare to the public leaderboard and iterate.
5. Very important — incorrect format gets rejected automatically.

Chapter 10 - Communicating results

10.1 Summary

- Communication turns results into action.
- Visualise key findings with clear, honest charts.
- Tell a story: problem → approach → findings → recommendations.
- Use the right chart for the data (bar, line, scatter, histogram).
- Highlight uncertainty, not just point estimates.
- Keep the audience in mind: executives vs peers need different detail levels.

10.2 Concepts

Storytelling with data

Element	Purpose
Title	States the key takeaway
Axes labels	Explain what each axis represents
Source	Cite data origin
Uncertainty	Error bars, confidence intervals
Comparison	Baselines or before/after

10.3 Examples

Example 1 - Summary table

Python

```python
# ch10_summary.py
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
import warnings
warnings.filterwarnings("ignore")

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["IsAlone"] = (df["FamilySize"] == 1).astype(int)
df["Title"] = df["Name"].str.extract(r", ([A-Za-z]+)\.")
df["Title"] = df["Title"].replace(["Lady", "Countess"], "Rare")
df["Age"] = df["Age"].fillna(df.groupby("Title")["Age"].transform("median"))

features = ["Pclass", "Sex", "Age", "Fare", "FamilySize", "IsAlone", "Title"]
X = df[features].copy()
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})
X["Title"] = X["Title"].map({"Mr": 1, "Miss": 2, "Mrs": 3, "Master": 4, "Rare": 5})
y = df["Survived"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "Gradient Boosting": GradientBoostingClassifier(random_state=42)
}

results = []
for name, model in models.items():
    model.fit(X_train, y_train)
    cv_scores = cross_val_score(model, X, y, cv=5)
    test_acc = model.score(X_test, y_test)
    results.append({
        "Model": name,
        "CV Mean": f"{cv_scores.mean():.3f}",
        "CV Std": f"{cv_scores.std():.3f}",
        "Test Acc": f"{test_acc:.3f}"
    })

results_df = pd.DataFrame(results)
print("Model Comparison")
print(results_df.to_string(index=False))
```

output

```
Model Comparison
                Model CV Mean  CV Std Test Acc
    Logistic Regression  0.800  0.034    0.804
          Random Forest   0.826  0.018    0.821
      Gradient Boosting   0.826  0.033    0.832
```

Example 2 - Survival analysis by group

Python

```python
# ch10_survival.py
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

# Survival rate by group
survival_by_sex = df.groupby("Sex")["Survived"].mean()
survival_by_class = df.groupby("Pclass")["Survived"].mean()

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].bar(survival_by_sex.index, survival_by_sex.values, color=["steelblue", "salmon"])
axes[0].set_title("Survival Rate by Sex")
axes[0].set_ylabel("Rate")
axes[0].set_ylim(0, 1)

axes[1].bar(survival_by_class.index, survival_by_class.values, color="mediumseagreen")
axes[1].set_title("Survival Rate by Passenger Class")
axes[1].set_xlabel("Class")
axes[1].set_ylabel("Rate")
axes[1].set_ylim(0, 1)

plt.tight_layout()
plt.savefig("survival_by_group.png", dpi=150)
print("Saved survival_by_group.png")
print(f"\nFemale survival rate: {survival_by_sex['female']:.1%}")
print(f"Male survival rate: {survival_by_sex['male']:.1%}")
print(f"First class survival rate: {survival_by_class[1]:.1%}")
print(f"Third class survival rate: {survival_by_class[3]:.1%}")
```

output

```
Saved survival_by_group.png
Female survival rate: 74.2%
Male survival rate: 18.9%
First class survival rate: 62.8%
Third class survival rate: 25.8%
```

10.4 Common mistakes and best practices

Mistake	Best practice
Too much jargon	Know your audience
Hiding uncertainty	Show confidence intervals
Misleading scales	Start bar charts at zero
Too many charts	Focus on 3-5 key messages
No narrative	Lead with the takeaway

10.5 Practice exercises

1. Create a summary slide/table comparing your models.
2. Make a bar chart of survival rates by a demographic.
3. Write a 3-sentence executive summary of your findings.
4. Identify 3 key insights from your analysis.
5. What one thing would you do next?

10.6 Solutions

Exercises 1-2: Use the examples above as templates. 3-5: Personal summary based on results.

10.7 Mini test

Questions

1. What should you lead with in a data presentation?
2. What chart type is best for comparing categories?
3. When should you show confidence intervals?
4. What is the "pyramid principle"?
5. How many key messages should a presentation have?

Answer key and explanations

1. The key takeaway / recommendation.
2. A bar chart.
3. Whenever you're showing estimated values.
4. Start with the conclusion, then support it with evidence.
5. 3-5 is the generally accepted maximum for retention.

Chapter 11 - Ethics and bias

11.1 Summary

- Ethics is central to data science, not an afterthought.
- Bias can enter at every stage: collection, labelling, modelling, deployment.
- Common biases: selection bias, confirmation bias, measurement bias, algorithmic bias.
- Fairness metrics: demographic parity, equal opportunity, equalised odds.
- Transparency: explainable AI, documenting decisions.
- Privacy: anonymisation, differential privacy, consent.

11.2 Concepts

Types of bias

Bias	Description	Example
Selection bias	Data not representative	Surveying only website visitors
Confirmation bias	Favoring expected results	Ignoring contradictory evidence
Measurement bias	Faulty data collection	Broken sensor reading
Algorithmic bias	Model discriminates	Hiring model favouring one gender
Label bias	Historical bias in labels	Past loans denied to minorities

11.3 Examples

Example 1 - Checking for bias in the model

Python

```python
# ch11_bias.py
import pandas as pd
import warnings
warnings.filterwarnings("ignore")

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

# Survival rate by sex and class
print("Overall survival rate:", df["Survived"].mean())
print("\nSurvival by sex:")
print(df.groupby("Sex")["Survived"].mean())
print("\nSurvival by class:")
print(df.groupby("Pclass")["Survived"].mean())
print("\nSurvival by sex AND class:")
print(df.groupby(["Pclass", "Sex"])["Survived"].mean())

# Check if the model is fair
# If we train on all features, "Sex" might be predictive but ethically questionable
# A fairer approach: exclude Sex and see how accuracy changes
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

df["FamilySize"] = df["SibSp"] + df["Parch"] + 1

# With gender
features_with_gender = ["Pclass", "Sex", "Age", "Fare", "FamilySize"]
X1 = df[features_with_gender].copy()
X1["Sex"] = X1["Sex"].map({"male": 0, "female": 1})
X1["Age"] = X1["Age"].fillna(X1["Age"].median())
y = df["Survived"]

X_train, X_test, y_train, y_test = train_test_split(X1, y, test_size=0.2, random_state=42)
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
acc_with = accuracy_score(y_test, model.predict(X_test))

# Without gender
features_without = ["Pclass", "Age", "Fare", "FamilySize"]
X2 = df[features_without].copy()
X2["Age"] = X2["Age"].fillna(X2["Age"].median())

X_train2, X_test2, y_train2, y_test2 = train_test_split(X2, y, test_size=0.2, random_state=42)
model2 = RandomForestClassifier(random_state=42)
model2.fit(X_train2, y_train2)
acc_without = accuracy_score(y_test2, model2.predict(X_test2))

print(f"\nAccuracy WITH gender feature: {acc_with:.3f}")
print(f"Accuracy WITHOUT gender feature: {acc_without:.3f}")
print(f"Difference: {acc_with - acc_without:.3f}")
```

output

```
Overall survival rate: 0.384

Survival by sex:
Sex
female    0.786
male      0.314

Surv
```